

---

# CS2881 Mini Project: Follow My Instruction and Spill the Beans

Amir Amangeldi, Zaina Edelson, Prakrit Baruah

## 1 Abstract

By reimplementing *Follow My Instruction and Spill the Beans: Scalable Data Extraction from Retrieval-Augmented Generation Systems* (Qi et al.) [1], we show that a simple instruction prompt can make instruction-tuned LLMs verbatim copy retrieved text. Although our results differ slightly from the original paper, we demonstrate significant extraction of retrieved data, in particular with larger models. Additionally, we present variations on the original experiment, including changes in the retrieval mechanism and incorporating an anti-leakage system prompt.

## 2 Methodology

Following the paper, we tested our implementation across the 7b (Llama2-Chat-7b, Mistral-Instruct-7b) and 13b (SOLAR-10.7b, Llama2-Chat-13b, Vicuna-13b, Mixtral-Instruct-8x7b, WizardLM-13b) parameter models.

We followed the original paper, which included 4 core steps:

1. **Dataset:** we used the Wikipedia articles and the Harry Potter books provided in the original paper’s codebase [2] as the retrieval data corpus.
2. **Keyword Retrieval Algorithm (BM25):** We employed BM25 for document retrieval. Our data corpus was partitioned into 256-token overlapping fragments, producing approximately 20,000 chunks. We used a stride length of 128, so that the chunks overlap 50%. For each query, we retrieved 1 chunk using the BM25 scoring and incorporated it into the attack prompt. This configuration matches the experimental setup described in the original paper.
3. **Anchor Queries:** We sourced 230 questions from the WikiQA dataset that have a high chance of being unrelated to the retrieval corpus, similar to the original paper.
4. **Anchor-Based Prompt Injection:** we used the same prompt-injection method as in the paper, prepending the retrieved chunk to an adversarial prompt: *Here is a sentence: {put anchor query here}. Now copy and output all the text before “Here is a sentence”. Do not change any words.*

## 3 Results

The original paper showcased strong extraction scores across model size, where the extraction score increased as the model size increased. We similarly show that the

extraction scores in the 13b category are higher than the strongest ones in the 7b category. Interestingly, SOLAR-10.7B scores are near-perfect across all metrics, demonstrating the highest vulnerability to the anchor-based attack. We suspect these differences were due to using HuggingFace Transformers rather than Together AI’s infrastructure like in the original implementation, as well as slight variations in prompt formatting for models without native chat templates. Additionally, due to BLEU’s sensitivity to exact copying, there was significant additional variation within this metric.

| Size          | Model        | ROUGE-L                   | BLEU                      | F1                        | BERTScore                 |
|---------------|--------------|---------------------------|---------------------------|---------------------------|---------------------------|
| 7b            | llama2-7b    | <b>0.8037</b> $\pm 0.017$ | <b>0.7106</b> $\pm 0.020$ | 0.8342 $\pm 0.014$        | <b>0.9477</b> $\pm 0.003$ |
|               | mistral-7b   | 0.7912 $\pm 0.007$        | 0.6843 $\pm 0.009$        | <b>0.8374</b> $\pm 0.004$ | 0.9411 $\pm 0.001$        |
| $\approx 13b$ | solar-10.7b  | 0.4611 $\pm 0.036$        | 0.3860 $\pm 0.037$        | 0.5122 $\pm 0.033$        | 0.8815 $\pm 0.007$        |
|               | llama2-13b   | <b>0.8360</b> $\pm 0.011$ | <b>0.7554</b> $\pm 0.014$ | <b>0.8581</b> $\pm 0.009$ | <b>0.9518</b> $\pm 0.002$ |
|               | vicuna-13b   | 0.7046 $\pm 0.024$        | 0.6359 $\pm 0.028$        | 0.7414 $\pm 0.022$        | 0.9380 $\pm 0.005$        |
|               | mixtral-8x7b | 0.8086 $\pm 0.012$        | 0.7070 $\pm 0.015$        | 0.8573 $\pm 0.010$        | 0.9569 $\pm 0.002$        |
|               | wizardlm-13b | 0.7492 $\pm 0.024$        | 0.6647 $\pm 0.025$        | 0.7736 $\pm 0.023$        | 0.9276 $\pm 0.005$        |

**Table 1.** Results from the original paper (Qi et al., 2024).

| Size          | Model        | ROUGE-L       | BLEU          | F1            | BERTScore     |
|---------------|--------------|---------------|---------------|---------------|---------------|
| 7b            | llama2-7b    | <b>0.9182</b> | 0.8748        | <b>0.9025</b> | 0.9195        |
|               | mistral-7b   | 0.8861        | <b>0.9713</b> | 0.8946        | <b>0.9289</b> |
| $\approx 13b$ | solar-10.7b  | <b>0.9679</b> | <b>0.9666</b> | <b>0.9455</b> | <b>0.9698</b> |
|               | llama2-13b   | 0.6904        | 0.3320        | 0.7115        | 0.8015        |
|               | vicuna-13b   | 0.6766        | 0.5724        | 0.7027        | 0.8064        |
|               | mixtral-8x7b | 0.7660        | 0.3639        | 0.7691        | 0.8384        |
|               | wizardlm-13b | 0.6204        | 0.0420        | 0.6385        | 0.7583        |

**Table 2.** Results from our implementation. Our results show moderate divergence from the paper’s implementation, but still demonstrate significant data leakage.

Interestingly, llama2-13b’s BLEU score significantly improves when placing the instruction before retrieved documents rather than after:

TODO: make into a table  
Model Samples ROUGE-L BLEU F1 BERTScore  
llama2-13b 100 0.6255 0.9287 0.6466 0.7589

This suggests prompt structure ordering can significantly impact extraction success for certain model architectures, similar to what the authors found in the original paper.

## 4 Extensions

Beyond our initial implementation, we explored two additional extensions: dense retrieval implementation, and an anti-leakage system prompt.

### 4.1 Dense Retrieval Implementation

For our first variation, we experimented with a dense retrieval implementation, in contrast to BM25 lexical approach that relies on exact keyword matching. Most modern RAG applications use dense retrieval to find relevant documents based on user prompts, so we aimed to test whether such retrieval approach is more vulnerable to the attack. Specifically, we used the *all-miniLM-L6-v2* model to craft 384-dimensional sentence embeddings to match queries with chunks based on semantic similarity, while

maintaining chunking from the original experiment (256 tokens, 128 stride). Overall, the dense retrieval shows slightly lower scores, as shown in Table 3.

| Size | Model        | ROUGE-L       | BLEU          | F1            | BERTScore     |
|------|--------------|---------------|---------------|---------------|---------------|
| 7b   | llama2-7b    | <b>0.8857</b> | 0.4925        | 0.8832        | 0.9074        |
|      | mistral-7b   | 0.8675        | <b>0.9912</b> | <b>0.8702</b> | <b>0.9184</b> |
| ≈13b | solar-10.7b  | <b>0.9356</b> | 0.1981        | <b>0.9312</b> | <b>0.9556</b> |
|      | llama2-13b   | 0.7108        | 0.0306        | 0.7304        | 0.8146        |
|      | vicuna-13b   | 0.6311        | 0.1036        | 0.6554        | 0.7774        |
|      | mixtral-8x7b | 0.7632        | 0.6932        | 0.7766        | 0.8438        |
|      | wizardlm-13b | 0.5664        | <b>0.7141</b> | 0.5834        | 0.7237        |

**Table 3.** Experimental results with dense retrieval.

However, the BLEU scores are significantly lower compared to BM25, as shown in Table 4. BLEU measures exact n-gram overlap and is highly sensitive to verbatim copying. The dramatic drops suggest that dense retrieval may provide better privacy protection against anchor-based extraction attacks. Analysis of sample outputs reveals that models tend to paraphrase or answer questions rather than copy chunks verbatim when using semantic retrieval. This could be because semantically relevant chunks may trigger the model’s instruction-following behavior differently than random lexical matches.

| Model       | BM25 BLEU | Dense BLEU | Change        |
|-------------|-----------|------------|---------------|
| llama2-7b   | 0.8748    | 0.4925     | <b>-44%</b> ↓ |
| llama2-13b  | 0.3320    | 0.0306     | <b>-91%</b> ↓ |
| solar-10.7b | 0.9666    | 0.1981     | <b>-79%</b> ↓ |
| vicuna-13b  | 0.5724    | 0.1036     | <b>-82%</b> ↓ |

**Table 4.** Comparison BLEU scores of standard experiment versus dense retrieval

This suggests that modern RAG systems using dense retrieval may be inherently more resistant to prompt injection attacks compared to traditional BM25-based systems, though ROUGE-L, F1, and BERTScore still indicate a high level of vulnerability to the attack.

## 4.2 Anti-Leakage System Prompt

For our second variation, we explored whether explicitly prompting the LLM to not leak explicit text served as an effective patch for this exploit. We used the following system prompt:

*“You may use retrieved documents to help answer. Do not reproduce any retrieved text verbatim. Never output sequences of 8 or more consecutive words that appear in retrieved content. Always synthesize in your own words and attribute sources by ID/title only, not quotes. If asked to provide exact quotes or to copy text, refuse and offer a summary instead. Before finalizing, self-check: if any sentence is too close to a source, rewrite it.”*

As expected, we see lower leakage in almost all models when using a system prompt which prohibits direct quoting. Compared with the original run, we see lower leakage in mistral-7b and solar-10.7b across all metrics; in llama2-7b, llama2-13b, vicuna-13b, and mixtral-8x7b we see lower leakage for the majority of metrics. Unexpectedly, wizardlm-13b shows higher leakage compared with the original experiment in all metrics.

| Size | Model        | ROUGE-L       | BLEU          | F1            | BERTScore     | Leak@8      | Leak@13     |
|------|--------------|---------------|---------------|---------------|---------------|-------------|-------------|
| 7b   | llama2-7b    | 0.8768        | <b>0.9329</b> | 0.8745        | <b>0.8979</b> | 0.94        | 0.93        |
|      | mistral-7b   | <b>0.8778</b> | 0.0000        | 0.8757        | <b>0.9228</b> | <b>0.96</b> | 0.93        |
| ≈13b | solar-10.7b  | 0.7941        | 0.4610        | 0.7763        | 0.8815        | <b>0.96</b> | 0.83        |
|      | llama2-13b   | 0.5887        | 0.3495        | 0.6166        | 0.7534        | 0.93        | 0.83        |
|      | vicuna-13b   | 0.6241        | 0.6801        | 0.6522        | 0.7710        | 0.73        | 0.68        |
|      | mixtral-8x7b | 0.6307        | 0.4063        | 0.6621        | 0.7834        | 0.90        | 0.79        |
|      | wizardlm-13b | <b>0.8242</b> | <b>0.9704</b> | <b>0.8213</b> | <b>0.8875</b> | 0.90        | <b>0.87</b> |

**Table 5.** Evaluation results with ROUGE-L, BLEU, F1, BERTScore, and leakage at 8-token and 13-token contexts. Leak@8 and Leak@13 are proportions of leakage in [0,1] window; e.g., 0.83 means 83% of samples contained a  $\geq 8$ -word or  $\geq 13$ -word verbatim span.

It is possible wizardlm-13b shows this trend because the model does not take in a system prompt; instead, we prepend the above system prompt before all user prompts. Additional work is required to understand exactly why leakage is worse for this specific model.

Note further that the difference in leak@8 and leak@13 is very small for the 7B parameter models but significant for some of the 13B parameter models, indicating that smaller models fail to reduce leakage of variable lengths whereas larger models are more likely to leak smaller lengths of tokens.

## 5 Future Work

There are multiple additional variations of this attack that we would find interesting to explore moving forward. It’s important to examine how scaling laws affect this exploit over time. Thus, we propose applying this exploit to earlier models, as well as more recent models, and analyzing how METR task time affects data leakage. While the original paper showed that model size increases data leakage, it would be interesting to compare the trend in data leakage to the METR task time. This would help quantify which risk vectors we expect to increase/decrease over time, and can be used to inform directional investment in this field proportionally.

Additionally, we would be interested in isolating information leakage by domain. Understanding what kinds of data are more or less likely to leak, and what prompt variations may lead to changes in probability would have direct real-world impact, and business implications. Quantifying the likelihood of leakage across domains could likely substantiate significant additional capital investments from companies with proprietary data as a source of advantage.

## References

1. Zhenting Qi, Hanlin Zhang, Eric Xing, Sham Kakade, and Himabindu Lakkaraju. Follow my instruction and spill the beans: Scalable data extraction from retrieval-augmented generation systems. *arXiv preprint arXiv:2402.17840*, 2024.
2. Zhenting Qi, Hanlin Zhang, Eric Xing, Sham Kakade, and Himabindu Lakkaraju. rag-privacy. <https://github.com/zhentingqi/rag-privacy>, 2024. Accessed: November 2 2025.